Ideation Phase

Brainstorm & Idea Prioritization Template

Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

Step-1: Team Gathering, Collaboration and Select the Problem Statement

The team identified the need to protect student/institution records in ServiceNow from unauthorized access. The selected problem is to restrict access to records based on the user's role and the Branch field, while allowing administrators full access.

Step-2: Brainstorm, Idea Listing and Grouping

Brainstormed ideas:

- Role-based access control: Use custom roles such as bb1, bb2, bb3 and bb4 to control Read, Create, Write and Delete operations.
- Branch-based record restriction: Allow EEE users to access records where Branch is EEE and prevent unauthorized users from viewing them.
- Script-controlled ACL: Use an ACL script to check whether the current user is an administrator or has the required custom role.
- Operation-specific ACLs: Create separate ACLs for read, create, write and delete operations.
- Administrator override: Allow users with the admin role to retain full access to all records.

Step-3: Idea Prioritization

The team prioritized the ideas based on security relevance, implementation simplicity and ability to demonstrate record-level access control.

Idea	Priority	Reason
Script-controlled ACL	High	Directly enforces access decisions in ServiceNow.
Branch-based restriction	High	Protects EEE records from unauthorized viewing.
Operation-specific ACLs	High	Clearly demonstrates Read/Create/Write/Delete permissions.
Role-based access	High	Provides controlled permissions for different users.
Admin override	Medium	Ensures administrators retain full access.

Ideation Phase

Define the Problem Statements

Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	2 Marks

Customer Problem Statement

The following statements define the access-control challenges faced by users and administrators working with institutional records in ServiceNow.

Problem Statement (PS)	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	an EEE user	view the student/institution records relevant to my branch	I should not be able to see records belonging to other branches	records contain branch-specific information and access must be restricted	concerned about unauthorized access to records
PS-2	a ServiceNow administrator	control who can read, create, edit and delete institution records	users may have permissions that are broader than required	different operations require different roles and ACL rules	responsible for maintaining secure and controlled access

Ideation Phase

Empathize & Discover

Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

Empathy Map Canvas – Primary User: EEE User

The empathy map captures the user's perspective while accessing institution/student records through ServiceNow.

SAYS	"I need to view the records related to my branch." "I should not access records that I am not authorized to see."
THINKS	Access should be limited according to my role. The system should protect sensitive student information.
DOES	Logs in to ServiceNow, opens the Institution Details records and views records permitted by the ACL.
FEELS	Concerned about data privacy and unauthorized access. More confident when access is controlled automatically.
NEEDS / GOALS	Branch-specific record access, clear permissions, secure data handling and reliable ACL enforcement.

Empathy Map – Secondary User: Administrator

SAYS	"I need full access to manage institution records."
THINKS	Permissions should be controlled through roles and ACLs.
DOES	Creates/manages records and verifies Read, Create, Write and Delete access.
FEELS	Responsible for protecting records while allowing authorized work.
NEEDS / GOALS	Full administrative access, controlled user permissions and secure record-level access.

Key insight: The solution should combine user roles, ACL operations and branch-based restrictions so that users receive only the access required for their responsibilities.

Project Design Phase-II
Data Flow Diagram & User Stories

Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

Data Flow Diagrams:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored.

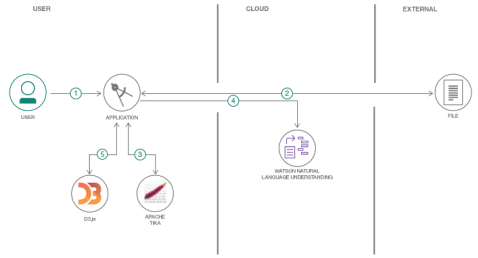
Example: DFD Level 0 (Industry Standard)

Admin / Authorized User → ServiceNow Institution Details Table → ACL

Data Store: u_institution_details
Fields: Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, Description
Roles: bb1 (Read), bb2 (Create), bb3 (Write), bb4 (Delete), admin (Full Access)

DFD – Level 0 (Script-Controlled ACL)

Flow



1. User configures credentials for the Watson Natural Language Understanding service and starts the app.
2. User selects data file to process and load.
3. Apache Tika extracts text from the data file.
4. Extracted text is passed to Watson NLU for enrichment.
5. Enriched data is visualized in the UI using the D3.js library.

User Stories

Use the below template to list all the user stories for the product.

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
EEE User	Record Access	USN-1	As an EEE user with bb1, I can view EEE branch records.	Only EEE branch records are visible.	High	Sprint-1
EEE User	Record Access	USN-2	As a user without the required role, I cannot view protected records.	Access is denied.	High	Sprint-1
Administrator	Record Access	USN-3	As an administrator, I can view all Institution Details records regardless of branch.	Admin can view EEE, ECE and CSE records.	High	Sprint-1
EEE User	Create Record	USN-4	As a user with bb2, I can create a new Institution Details record.	New record can be created.	High	Sprint-1
Authorized User	Write Record	USN-5	As a user with bb3, I can edit permitted records.	Record can be modified.	High	Sprint-1

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Authorized User	Delete Record	USN-6	As a user with bb4, I can delete permitted records.	Record can be deleted.	High	Sprint-1
Administrator	Administration	USN-7	As an administrator, I can access records without the branch restriction.	Full access is retained.	High	Sprint-1
Security Administrator	ACL Management	USN-8	As a security administrator, I can configure record ACLs for the table.	ACLs are active and enforce the configured operations.	High	Sprint-1

Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	3 October 2026
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Team ID	SWTID-2026-5163
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User and Role Management	Create EEE test user; create bb1, bb2, bb3 and bb4 roles; assign roles to the user.
FR-2	Institution Details Records	Create u_institution_details table with student and branch-related fields; create records with ECE, EEE and CSE values.
FR-3	Access Control	Configure Read ACL with bb1 and Branch = EEE condition; use script to allow admin and bb1, and deny others.
FR-4	Record Operations	Configure Create, Write and Delete ACLs using bb2, bb3 and bb4 respectively; verify access for users with the required roles.
FR-5	Access Verification	Impersonate users and administrator to verify record visibility and permitted operations.

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

FR No.	Non-Functional Requirement	Description
NFR-1	Usability	ACL configuration and verification should be clear enough for administrators to manage roles and record access.
NFR-2	Security	Record access is controlled using ACLs, roles, a Branch data condition, and a script; unauthorized users are denied.
NFR-3	Reliability	The same ACL rules should consistently enforce Read, Create, Write and Delete permissions.
NFR-4	Performance	Access decisions should be evaluated during normal ServiceNow record operations without unnecessary processing in the ACL script.
NFR-5	Availability	The solution operates within the ServiceNow instance; availability depends on the configured ServiceNow environment.

NFR-6	Scalability	Role-based ACLs can be extended to additional users, roles, tables and field-based access rules as requirements grow.

Project Design Phase-II
Technology Stack (Architecture & Stack)

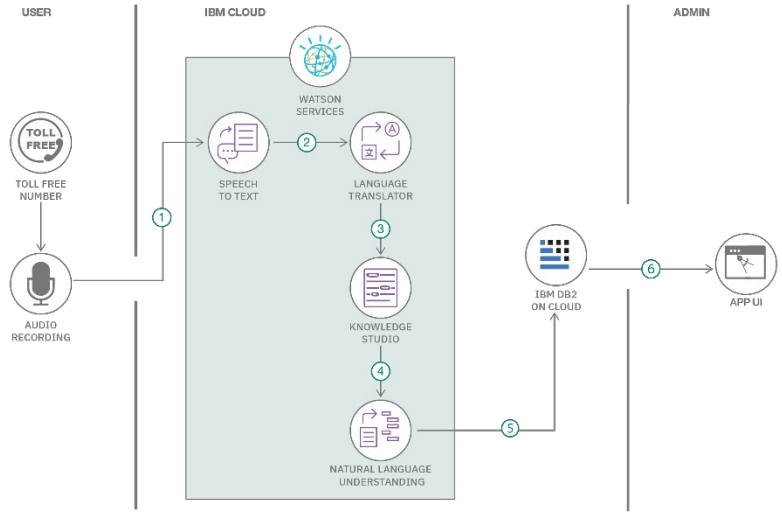
Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Technical Architecture: ServiceNow ACL-Based Record Security

Architecture flow: ServiceNow User → Roles → ACL Engine → Branch/Data Condition + Script → Institution Details Table → Allow / Deny



Guidelines:

- Include all the processes (As an application logic / Technology Block)
- Provide infrastructural demarcation (Local / Cloud)
- Indicate external interfaces (third party API's etc.)
- Indicate Data Storage components / services
- Indicate interface to machine learning models (if applicable)

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	ServiceNow interface used to access the Institution Details records and lists.	ServiceNow UI / Forms / Lists
2	Application Logic-1	Record-level Read access checks the required role and EEE branch condition.	ServiceNow ACL
3	Application Logic-2	Create access is controlled through the required role.	ServiceNow Create ACL
4	Application Logic-3	Write and Delete operations are controlled through separate role-based ACLs.	ServiceNow Write ACL / Delete ACL
5	Database	Stores Institution Details records and fields such as Branch, Student Name and contact details.	ServiceNow platform data store
6	Cloud Database	Cloud database service is not separately specified in the source lab.	Not specified
7	File Storage	No separate file-storage component is specified in the source lab.	Not specified
8	External API-1	No external API is required for the ACL demonstration.	Not specified
9	External API-2	No external API is required for the ACL demonstration.	Not specified
10	Machine Learning Model	The source lab does not use a machine-learning model; access is controlled by ACL rules and script logic.	Not applicable
11	Infrastructure (Server / Cloud)	Application runs in a ServiceNow instance; the source lab does not specify a separate local server configuration.	ServiceNow instance

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	No open-source application framework is specified in the source lab.	Not specified
2	Security Implementations	Record-level ACLs, custom roles bb1–bb4, admin full access, Security_admin elevation for ACL configuration, Branch = EEE data condition, and scripted allow/deny logic.	ServiceNow ACL / Roles / Script
3	Scalable Architecture	The role-based ACL model can be applied to additional tables, roles and field conditions as the application grows.	ServiceNow ACL framework
4	Availability	The solution is hosted within the ServiceNow instance; specific load-balancing or distributed-server configuration is not specified.	ServiceNow instance
5	Performance	The access decision uses role checks and a record data condition; no separate cache/CDN or request-rate target is specified.	ServiceNow ACL evaluation

References:

https://c4model.com/

https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/

https://www.ibm.com/cloud/architecture

https://aws.amazon.com/architecture

https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d

Planning Logic – Script-Controlled ACL

Basic Agile Terms

Sprint: A fixed period or duration in which the team works to complete a set of tasks.

Epic: A large project/task that is too big to complete in one sprint and is divided into smaller stories.

Story: A small task that is part of an Epic.

Story Point: A number representing the effort required to complete a story. The values used here are 1, 2, 3 and 5.

Sprint 1

User and Role Setup (Epic 1)

Create the test user with required details. – Create EEE User (USN1) – 2 Story Points

Create bb1, bb2, bb3 and bb4 roles and assign them. – Create Roles (USN2) – 3 Story Points

Table Creation (Epic 2)

Create the custom table u_institution_details. – Create Institution Details Table (USN3) – 3 Story Points

Add roll number, student, faculty, branch, email, phone and description fields. – Add Required Fields (USN4) – 3 Story Points

Create ECE, EEE and CSE records. – Create Sample Records (USN5) – 2 Story Points

Total Story Points in Sprint 1 = 2 + 3 + 3 + 3 + 2 = 13

Sprint 2

READ ACL (Epic 3)

Create READ ACL (USN6) – 5 Story Points

Configure ACL Script (USN7) – 3 Story Points

CREATE ACL (Epic 4)

Create CREATE ACL with bb2 (USN8) – 2 Story Points

WRITE ACL (Epic 5)

Create WRITE ACL with bb3 (USN9) – 2 Story Points

DELETE ACL (Epic 6)

Create DELETE ACL with bb4 (USN10) – 2 Story Points

Total Story Points in Sprint 2 = 5 + 3 + 2 + 2 + 2 = 14

Sprint 3

Security Verification (Epic 7)

Impersonate users and verify READ, CREATE, WRITE and DELETE permissions (USN11) – 5 Story Points

Verify EEE access and administrator full access (USN12) – 3 Story Points

Total Story Points in Sprint 3 = 5 + 3 = 8

Total Story Points and Velocity

Sprint 1 = 13

Sprint 2 = 14

Sprint 3 = 8

Total Story Points = 13 + 14 + 8 = 35

Number of Sprints = 3

Velocity = Total Story Points Completed / Number of Sprints

Velocity = 35 / 3 = 11.67 Story Points per Sprint

ACL Verification Logic

- bb1 role → READ access to the required EEE records.
- bb2 role → CREATE permission.
- bb3 role → WRITE/Edit permission.
- bb4 role → DELETE permission.
- Admin → full access regardless of branch.

Project Planning Phase – Script-Controlled ACL

Project Planning Template (Product Backlog, Sprint Planning, Stories, Story Points)

Date: 3 October 2026
Team ID: SWTID-2026-5163
Project Name: Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks: 5 Marks

Product Backlog, Sprint Schedule, and Estimation

The following backlog is prepared from the Script-Controlled ACL lab requirements: user/role creation, custom table setup, ACL creation and verification.

Sprint	Functional Requirement (Epic)	User Story Number	User Story / Task	Story Points	Priority
Sprint-1	User and Role Setup	USN-1	Create the EEE test user with required user details.	2	High
Sprint-1	User and Role Setup	USN-2	Create custom roles bb1, bb2, bb3 and bb4 and assign them to the test user.	3	High
Sprint-1	Table Creation	USN-3	Create the Institution Details table and configure its required fields.	3	High
Sprint-1	Table Creation	USN-4	Add Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description fields.	3	High
Sprint-1	Data Preparation	USN-5	Create multiple records with ECE, EEE and CSE branch values.	2	Medium
Sprint-2	READ ACL	USN-6	Create a record-level READ ACL for u_institution_details with bb1 and EEE branch condition.	5	High
Sprint-2	READ ACL	USN-7	Configure the ACL script so administrators have full access and unauthorized users are denied.	3	High
Sprint-2	CREATE ACL	USN-8	Create a CREATE	2	Medium

			ACL requiring the bb2 role.		
Sprint-2	WRITE ACL	USN-9	Create a WRITE ACL requiring the bb3 role.	2	Medium
Sprint-2	DELETE ACL	USN-10	Create a DELETE ACL requiring the bb4 role.	2	Medium
Sprint-3	Security Verification	USN-11	Impersonate users and verify READ, CREATE, WRITE and DELETE permissions.	5	High
Sprint-3	Security Verification	USN-12	Verify that EEE-role users access EEE records and administrators retain full access.	3	High

Project Tracker, Velocity & Sprint Schedule

Sprint	Total Story Points	Duration	Sprint Start Date	Sprint End Date (Planned)	Story Points Completed	Sprint Release Date (Actual)
Sprint-1	15	6 Days	01 Oct 2026	06 Oct 2026	15	06 Oct 2026
Sprint-2	16	6 Days	08 Oct 2026	13 Oct 2026	16	13 Oct 2026
Sprint-3	8	6 Days	15 Oct 2026	20 Oct 2026	8	20 Oct 2026

Velocity

Total Story Points = 15 + 16 + 8 = 39

Number of Sprints = 3

Velocity = Total Story Points Completed / Number of Sprints

Velocity = 39 / 3 = 13 Story Points per Sprint

Burndown Chart

A burndown chart can be used to track the remaining ACL project work against time. The planned work reduces as user/role setup, table creation, ACL configuration and verification tasks are completed.

Project Design Phase

Problem – Solution Fit Template

Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	2 Marks

Problem – Solution Fit

This canvas maps the users, access-control problems, existing approaches, constraints, behaviour, root cause and the proposed ServiceNow ACL solution.

1. CUSTOMER SEGMENT(S) – CS • ServiceNow administrators • EEE branch users • Users who need controlled access to institution/student records	**6. CUSTOMER CONSTRAINTS – CC** • Users must have the required role to perform an operation. • Access must respect branch-based restrictions. • Unauthorized users should not view or modify records. • Administrators need full access.	**5. AVAILABLE SOLUTIONS – AS** • ServiceNow ACLs • Role-based access control • Record-level read/write/create/delete permissions • Script-controlled ACL conditions
2. JOBS-TO-BE-DONE / PROBLEMS – J&P • Protect institution/student records from unauthorized access. • Allow EEE users to view permitted EEE records. • Control who can create, edit and delete records. • Give administrators complete access.	**9. PROBLEM ROOT CAUSE – RC** • A single user population may have different access requirements. • Without properly configured ACLs, unauthorized users may access or modify records. • Access must be evaluated using both user roles and record conditions.	**7. BEHAVIOUR – BE** • Users access records through ServiceNow lists/forms. • Users with required roles perform permitted operations. • Users without required roles are denied access. • Administrators access records without branch restriction.
3. TRIGGERS – TR • Need to protect sensitive student/institution information. • A user attempts to open a restricted record. • A user attempts to create, edit or delete a record. • An administrator needs full access for management.	**10. YOUR SOLUTION – SL** Implement script-controlled ServiceNow ACLs for the Institution Details table. Read: bb1 + Branch = EEE, with admin full access. Create: bb2. Write: bb3. Delete: bb4. The ACLs enforce record-level security for the required operations.	**8. CHANNELS OF BEHAVIOUR – CH** ONLINE: • ServiceNow list views • ServiceNow forms • Application Navigator OFFLINE: • No separate offline access is required; access is controlled inside ServiceNow.

Reference: Based on the uploaded ServiceNow Script-Controlled ACL lab document.

Project Design Phase
Proposed Solution Template

Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	2 Marks

Proposed Solution Template:

Project team shall fill the following information in the proposed solution template.

S.No.	Parameter	Description
1.	Problem Statement (Problem to be solved)	Unauthorized users may view, modify, create, or delete student records. The solution restricts record access based on user roles and the Branch field, while administrators retain full access.
2.	Idea / Solution description	Implement a ServiceNow script-controlled Access Control List (ACL) for the u_institution_details table. Roles bb1, bb2, bb3 and bb4 control Read, Create, Write and Delete operations. The Read ACL uses the condition Branch = EEE and a script to allow administrators full access and authorized bb1 users to access the permitted records.
3.	Novelty / Uniqueness	The solution combines role-based access control with a record-level Branch condition and a script-controlled decision. Permissions are separated by operation, so users receive only the access associated with their assigned roles.
4.	Social Impact / Customer Satisfaction	The approach helps protect sensitive student information from unauthorized viewing, modification, creation and deletion. Authorized users receive the permissions required for their tasks, while administrators retain full access.
5.	Business Model (Revenue Model)	The source project document does not specify a direct revenue model. The proposed solution is a ServiceNow access-control implementation for securing organizational records.
6.	Scalability of the Solution	The ACL model can be extended with additional roles, conditions and permissions for other

		tables, branches and user groups. The same Read, Create, Write and Delete security model can therefore be applied to larger organizational datasets.

Project Design Phase

Solution Architecture

Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value

Maximum Marks 4 Marks

Solution Architecture:

The solution architecture for this ServiceNow micro-project defines how Access Control Lists (ACLs), user roles, and record field values work together to control access to the Institution Details table. The architecture restricts access to records according to the Branch value and the roles assigned to users.

The custom table used in the solution is u_institution_details. It contains Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, and Description fields. The Branch field contains ECE, EEE, and CSE values.

The access-control layer uses four custom roles: bb1, bb2, bb3, and bb4. The READ ACL uses the bb1 role and an EEE branch condition. The script also allows administrators to retain full access. CREATE, WRITE, and DELETE operations are controlled progressively through bb2, bb3, and bb4 roles.

Solution Architecture Flow:

ServiceNow User

↓

User Role Validation (bb1 / bb2 / bb3 / bb4 / admin)

↓

u_institution_details Table

↓

Branch / Record Condition Evaluation

↓

ACL Decision

↓

Allow or Deny Operation

Access Control Design:

Operation	Required Role	Access Purpose
READ	bb1	View EEE branch records
CREATE	bb2	Create records

WRITE	bb3	Edit records
DELETE	bb4	Delete records

Goals:

- Restrict unauthorized users from viewing protected records.
- Allow users with the required roles to perform the permitted ACL operations.
- Allow administrators to retain full access.
- Demonstrate record-level security using READ, CREATE, WRITE, and DELETE ACLs.
- Evaluate user roles and record field values when making access decisions.

Figure 1: Solution architecture and access flow of the Script-Controlled ACL ServiceNow application

Reference: Script-Controlled ACL – Restrict Record Access Based on Field Value

USER ACCEPTANCE TESTING (UAT)

Script-Controlled ACL – Restrict Record Access Based on Field Value

Project Information

Field	Details
Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	10 Marks
Project Version	1.0
Testing Period	01 October 2026

Project Overview

This project demonstrates a script-controlled Access Control List (ACL) in ServiceNow. Record access is restricted according to user roles and the Branch field. EEE branch records are intended to be accessible to authorized EEE users, while administrators retain full access.

Testing Scope

The testing covers READ, CREATE, WRITE and DELETE ACLs for the u_institution_details table; role-based access using bb1, bb2, bb3 and bb4; Branch values ECE, EEE and CSE; and administrator access.

Testing Environment

Field	Details
Application	ServiceNow
Table	u_institution_details
User Roles	bb1, bb2, bb3, bb4, admin
Record Field Used	Branch
Branch Values	ECE, EEE, CSE

Test Cases

Test Case ID	Test Scenario	Test Steps	Expected Result	Actual Result	Pass/Fail
TC-001	Read EEE records with bb1	Impersonate bb1 user; open u_institution_details list	Authorized EEE records are visible	To be recorded	
TC-002	Read without required role	Impersonate user without bb1; open list	Records are not accessible	To be recorded	
TC-003	Create with bb2	Log in as user with bb1+bb2; click New; save record	Create operation is available	To be recorded	
TC-004	Write with bb3	Use bb1+bb2+bb3; edit an EEE record	Record can be edited	To be recorded	
TC-005	Delete with bb4	Use bb1+bb2+bb3+bb4; delete an EEE record	Record can be deleted	To be recorded	
TC-006	Admin access	Impersonate admin;	Admin can	To be recorded	

		open records	access records regardless of branch		

Bug Tracking

Bug ID	Bug Description	Steps to Reproduce	Severity	Status	Additional Feedback
BG-001	To be recorded during testing	Record exact steps if ACL fails	Low/Medium/High	Open/In Progress/Closed	

Sign-off

Field	Details
Tester Name	
Date of Test Completion	
Signature	

Notes: Test cases should cover positive and negative scenarios. Record detailed feedback, severity, status and reproduction steps for defects. Obtain required project sign-off before deployment.

TABLEAU – MODEL PERFORMANCE TEST

Script-Controlled ACL – Restrict Record Access Based on Field Value

Field	Details
Date	3 October 2026
Team ID	SWTID-2026-5163
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	10 Marks

Model Performance Testing

The project is a ServiceNow ACL security implementation. Tableau-style performance documentation is adapted here to record the data/security elements relevant to the ACL implementation.

S.No.	Parameter	Values / Evidence
1	Data Rendered	Institution Details records containing Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number and Description.
2	Data Preprocessing	Records created with different Branch values: ECE, EEE and CSE.
3	Utilization of Filters	Access is logically restricted using the Branch condition (EEE) and user roles.
4	Calculation Fields Used	Not applicable to this ACL project.
5	Dashboard Design	Not applicable; ServiceNow list/form views are used.
6	Story Design	Not applicable; ACL verification is performed through user impersonation and record access.

Verification evidence: impersonate authorized and unauthorized users and compare the records and operations available to each role.

Script-Controlled ACL – FSD Project Documentation

Restrict Record Access Based on Field Value

1. Introduction

Project Title: Script-Controlled ACL – Restrict Record Access Based on Field Value

Team ID: SWTID-2026-5163

Team Size: 4

Team Leader: Bhuvaneshwari N

Team Members: Harini T, Varshini C, Esther S

Platform: ServiceNow

Project Type: Access control and record-level security lab / micro project

2. Project Overview

Purpose: This project demonstrates how to implement script-controlled Access Control Lists (ACLs) to restrict users from viewing and modifying records unless they meet specific conditions. The scenario focuses on EEE branch records, while administrators retain full access.

Main Features:

- Creation of a test EEE user.
- Creation of custom roles bb1, bb2, bb3 and bb4.
- Creation of the custom Institution Details table.
- Branch field with ECE, EEE and CSE choices.
- Record-level READ ACL with Branch = EEE condition.
- CREATE ACL controlled by bb2.
- WRITE ACL controlled by bb3.
- DELETE ACL controlled by bb4.
- Administrator full access through the ACL script.
- Verification through user impersonation.

3. Architecture

3.1 User and Role Architecture

User: A test user named EEE User is created.

Custom Roles: bb1, bb2, bb3 and bb4.

- bb1 – Read access
- bb2 – Create access
- bb3 – Write/Edit access
- bb4 – Delete access

3.2 Table Architecture

Table: Institution Details

Table Name: u_institution_details

- Student Roll Number – Auto Number
- Student Name – Reference – User
- Faculty Name – Reference – User
- Branch – Choice (ECE, EEE, CSE)
- Email – String
- Phone Number – String
- Description – Multi String

3.3 ACL Architecture

- READ ACL – bb1 role and Branch = EEE condition.
- CREATE ACL – bb2 role.
- WRITE ACL – bb3 role.
- DELETE ACL – bb4 role.
- Administrator access – allowed by the script.

4. Setup Instructions

4.1 Prerequisites

- A ServiceNow instance with administrator access.
- Security_admin role must be elevated for ACL creation.
- Access to User Administration, System Definition and System Security modules.

4.2 User Creation

1. Log in to the ServiceNow instance with admin access.
2. Navigate to User Administration → Users.
3. Click New.
4. Create the EEE User.
5. Save the user record.

4.3 Role Creation

6. Navigate to User Administration → Roles.
7. Click New.
8. Create the role bb1.
9. Repeat the process for bb2, bb3 and bb4.
10. Assign the required roles to the EEE User.

4.4 Table Creation

11. Navigate to System Definition → Tables.
12. Click New.
13. Create the Institution Details table with name u_institution_details.
14. Add the fields specified in Section 3.
15. Save the table.
16. Create multiple records with different branch values such as ECE, EEE and CSE.

5. Folder Structure

A software folder structure is not specified in the source document because this project is implemented as a ServiceNow configuration and ACL lab rather than a client/server code project.

- ServiceNow User Administration – Users and Roles
- System Definition – Custom Table and Fields
- System Security – Access Control (ACL)
- Institution Details – Records

6. Running the Application

The source document does not specify application start commands. The project is tested inside the ServiceNow instance.

17. Log in to the ServiceNow instance.
18. Create or use the configured users and roles.
19. Open the u_institution_details list.
20. Impersonate the required user to verify access.
21. Compare the user's available operations with the configured ACLs.

7. API Documentation

No backend API endpoints are specified in the source document. Access is controlled through ServiceNow record ACLs.

ACL Operations: READ, CREATE, WRITE and DELETE

Protected Table: u_institution_details

8. Authentication

The source document uses ServiceNow users and roles for access control. The lab focuses on authorization through ACLs rather than documenting an independent authentication mechanism.

Authorization model: Role-based and condition/script-controlled access.

- bb1 controls READ access.
- bb2 controls CREATE access.
- bb3 controls WRITE access.
- bb4 controls DELETE access.

- Administrators receive full access through the script.

8.1 READ ACL Script

```
(function () {
    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }

    // Allow only EEE branch users to see EEE records
    if (gs.hasRole('bb1')){
        return true;
    }

    // Deny access for all others
    return false;
})();
```

9. User Interface

The source document refers to the ServiceNow user interface for creating users, roles, tables, ACLs and records. It also specifies opening the Student Records list using the table list URL u_institution_details.list.

- User Administration → Users
- User Administration → Roles
- System Definition → Tables
- System Security → Access Control (ACL)
- Institution Details / Student Records list

10. Testing

Testing Method: User impersonation and ACL verification.

10.1 READ ACL Testing

- User with bb1 role can view EEE branch records.
- User without the required role cannot view records.
- Admin user can view all records regardless of branch.

10.2 CREATE ACL Testing

- User with bb1 and bb2 roles can view EEE branch records and use the New button.

10.3 WRITE ACL Testing

- User with bb1, bb2 and bb3 roles can view and edit EEE branch records along with the New button.

10.4 DELETE ACL Testing

- User with bb1, bb2, bb3 and bb4 roles can see, create, edit and delete EEE branch records.

11. Screenshots or Demo

The source document provides verification steps involving user impersonation and the Student Records list. Actual screenshots are not included in the source content provided.

- EEE User impersonation and Student Records list.
- Admin impersonation and Student Records list.
- ACL configuration screens.
- Verification of New, Edit and Delete access.

12. Known Issues

No specific bugs or implementation issues are documented in the source document.

Important configuration note: The READ ACL uses a Branch = EEE data condition together with the bb1 role and an advanced script. The source document should be followed exactly when configuring the ACL.

13. Future Enhancements

The source document does not specify future enhancements. Based strictly on the documented ACL structure, possible extensions are not defined in the source and therefore are not added here.

Project Outcome

By completing this lab, learners gain an understanding of how script-controlled READ, WRITE, CREATE and DELETE ACLs enforce record-level security, how user roles and record field values are evaluated together to make access decisions, and how ACLs protect sensitive data from unauthorized viewing, modification, creation and deletion across forms, lists and Service Portal interfaces.